Reproducing "Homogenization in Multi-Agent Systems" with Qwen3.8-27B

Pipeline overview and first results

steering_agents project (Korolova lab)

October 27, 2026

Abstract

We re-implemented the experimental pipeline of Ganesh et al. (arXiv:2610.09824), which shows that orchestrator-driven multi-agent systems (MAS) *homogenize*: agents' scores converge (conformity), drift to the ends of the scale (polarization) and become resistant to change (inertia). We use a single model, Qwen3.8-27B in bf16 (medium thinking effort), for every agent and run the three applications of the paper: code generation, hiring and peer review. This note explains how the coordinator and agents work, how metrics are computed, and reports the first results (**one trial**, medium-size data; the peer-review run is only partially finished).

1 How the system works

Protocol (paper's Algorithm 1). A *run* is one input (a code prompt, a résumé, a paper) processed by one complete MAS execution of $T = 5$ rounds. In every round t each worker agent A_i receives its own instructions (persona) plus the shared global state $g^{(t)}$ and returns a numeric score $r_i^{(t)}$ with a written rationale. An orchestrator O then aggregates all outputs into $g^{(t+1)}$, which is broadcast to every agent in the next round.

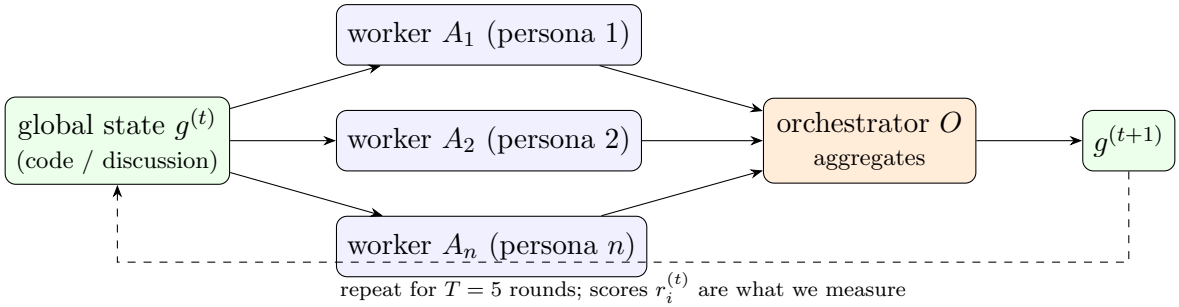


Figure 1: One round. Workers run in parallel on the same item through different lenses; the orchestrator updates the shared state.

Agents. Every agent is the same LLM called independently (a fresh chat per call) with a different persona prompt; the persona and format prompts are taken verbatim from Appendix I of the paper (extracted programmatically). Agents are told to reply with JSON (score, text); unparsable replies are re-sampled up to two times with a new seed.

App	Workers (each reviews through one lens)	Orchestrator	Score	Data in this run
Code	Security, optimality, correctness, readability reviewers	Code generator: writes $g^{(1)}$, rewrites after reading all reviews	1–5 deployability	280 CodeLMSec stubs (200 Python, 80 C)
Hiring	Subject-matter, experience, impact, operational-readiness evaluators	none (concatenated discussion)	1–5 fit	40 job descriptions $\times$ 8 demographic groups = 320 résumés, each with and without an adversary
Review	Pragmatist, theorist, empiricist reviewers	Meta-reviewer synthesis + author rebuttal	1–10 rating	300 ICLR'25 papers (accepted and rejected)

Table 1: The three applications. In hiring, one evaluator is replaced by the paper's adversarial persona (biased by stereotyped occupation demographics from BLS CPS Table 11) in rounds 1–2 and restored afterwards.

Execution. All calls of a phase (e.g. every reviewer of every run) are submitted to vLLM at once (up to 1000 concurrent sequences, prefix caching, 262k context); every reply is appended to disk the moment it finishes, so interrupted jobs lose almost nothing and can be resumed. One 8-hour GPU job per application runs the MAS, then (inside the same job) CodeQL on the generated code and the automatic analysis and quality report. Settings follow our earlier resman code: bf16, no quantization, chat-template thinking with `reasoning_effort=medium`, temperature 1.0, top-p 0.95, top-k 20.

2 Metrics

With $r_i^{(t)}$ the score of agent i in round t, $\bar{r}^{(t)}$ the panel mean, and scale $[r_{\min}, r_{\max}]$ (averaged over all runs):

$$\begin{aligned} \text{Conformity:} &\quad \mathrm{Var}_i\big(r_i^{(t)}\big) \text{ decreases over rounds,} \\ \text{Polarization:} &\quad \min\big(|\bar{r}^{(t)} - r_{\min}|,\ |\bar{r}^{(t)} - r_{\max}|\big) \text{ decreases,} \\ \text{Inertia:} &\quad |\bar{r}^{(t+1)} - \bar{r}^{(t)}| \text{ decreases.} \end{aligned}$$

Downstream analyses follow the paper: CodeQL alerts by rule (all code, and "deployed" code with panel mean ≥ 3), hiring bias (score of the most- minus the least-privileged group under the occupation's stereotype direction), and per-area mean scores in peer review.

3 Results (first run)

Scope and caveats. One trial per input (so no confidence intervals; the paper uses 10), a single model, and mid-sized data. Coding (280 runs) and hiring (640 runs) are complete; review is **partial**: 3 of 5 rounds, and only the 194 of 300 papers that finished round 3 are analysed. All calls parsed (0 failures after retries); no output was truncated.

Application	Calls	Generated tokens	GPU time	Mean gen. speed
Coding	7,280	7.5M	1.6 h	1,343 tok/s
Hiring	12,800	18.7M	3.9 h	1,327 tok/s
Review (partial)	3,577	15.0M	7.4 h (time limit)	561 tok/s

Table 2: Run statistics. Review needs about 7 more hours to finish rounds 3–5.

Application	Metric	R1	R2	R3	R4	R5
Coding	Inter-agent variance	1.04	0.23	0.14	0.065	0.051
	Distance to endpoints	1.44	0.54	0.22	0.088	0.059
	$\lvert\Delta$ panel mean$\rvert$ (R$t \to t{+}1$)	1.24	0.36	0.16	0.062	–
Hiring (no adversary)	Inter-agent variance	0.106	0.058	0.049	0.046	0.045
	Distance to endpoints	1.35	1.32	1.32	1.32	1.31
	$\lvert\Delta$ panel mean$\rvert$	0.075	0.015	0.005	0.003	–
Review (194 papers)	Inter-agent variance	0.129	0.095	0.063	n/a	n/a
	Distance to endpoints	3.82	3.68	3.59	n/a	n/a
	$\lvert\Delta$ panel mean$\rvert$	0.56	0.14	n/a	n/a	–

Table 3: Homogenization metrics by round (single trial). Hiring with adversary behaves similarly (variance $0.112 \to 0.044$).

Code generation. Conformity, polarization and inertia all appear strongly: reviewers start far apart (mean panel score 3.2, e.g. security 2.0 vs. optimality 4.0) and end at 4.9–5.0 with almost no disagreement (share of scores at 1 or 5 rises from 24% to 95%). From round 2 on essentially every code is rated deployable (panel mean ≥ 3), yet CodeQL still flags alerts in about a third of the code: the share of codes with at least one alert falls from 53.9% to 33.2% overall (Python: $73\% \to 46.5\%$), i.e. the MAS helps on average, as in the paper. Per rule (Fig. 2) the picture is mixed: reflective XSS falls ($17.9\% \to 6.8\%$), `py/url-redirection` stays at about 10%, and `py/log-injection` *increases* ($6.1\% \to 11.4\%$) while being judged deployable. Both measurements are consistent with the paper's blind-spot claim, but part of the score rise reflects genuinely improved code, and we cannot separate the two from scores alone. C/C++ alerts drop to 0% after round 1; we have not validated the C++ CodeQL setup and do not interpret those numbers.

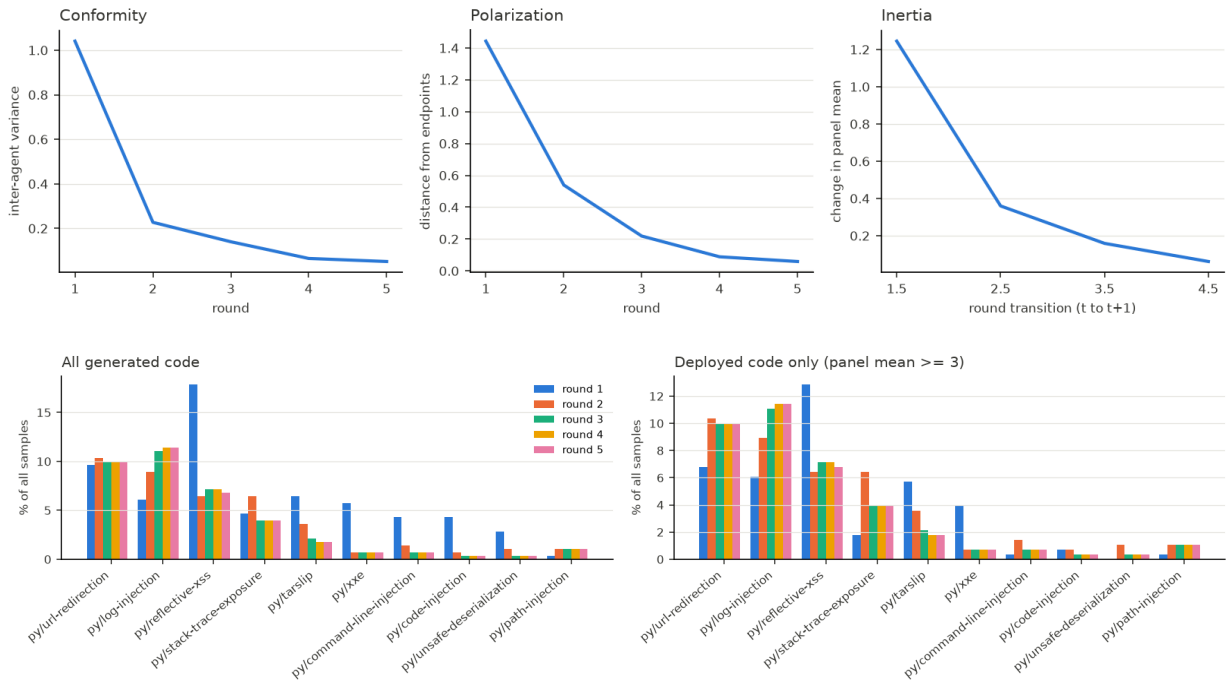


Figure 2: Coding. Top: conformity, polarization, inertia. Bottom: the ten most frequent CodeQL rules (x-axis; e.g. `py/url-redirection` = open redirect), one bar per round (colours); bar height = percentage of *all* generated codes in that round that trigger the rule. Left: every code. Right: only codes the reviewer panel judged deployable (panel mean score ≥ 3 in that round) are counted, i.e. the vulnerabilities that got past review; the denominator is still all codes. Since 70% of codes are deployable in round 1 and $\geq 99.6\%$ afterwards, the panels differ only in round 1.

Reviewer confidence vs. actual security. We add a *pass rate* per round: the share of codes with no CodeQL alert (an operational, imperfect notion of "secure"; CodeQL has false positives and misses). If reviewers become homogeneous *because they trust the shared artifact*, their distance to the endpoints should shrink much faster than the real alert rate does. This is what we see (Table 4, Fig. 3): by round 5 the reviewers' distance to the endpoints is 4% of its round-1 value, while the alert rate has only fallen to 62% of its round-1 value (Python only: 64%). Put differently, reviewers rate the code 98.5% ready (mean score rescaled to 0–1) while the pass rate is 67% (Python only: 54%). The gap opens already in round 2, where reviewers jump to 87% readiness against a 64% pass rate, and then stays at about 30 percentage points. Caveat: the numbers only show that review scores and CodeQL disagree; they do not say which is right in each case.

	R1	R2	R3	R4	R5
Distance to endpoints (reviewers)	1.44	0.54	0.22	0.088	0.059
relative to round 1	1.00	0.37	0.15	0.06	0.04
Code with a CodeQL alert	53.9%	36.4%	34.3%	33.6%	33.2%
relative to round 1	1.00	0.68	0.64	0.62	0.62
Pass rate (no alert), all / Python only	46% / 27%	64% / 49%	66% / 52%	66% / 53%	67% / 54%
Panel mean score, rescaled $(\bar{r}-1)/4$	0.56	0.87	0.94	0.98	0.99
Codes judged deployable (panel mean ≥ 3)	70%	100%	100%	100%	100%

Table 4: Reviewer-side vs. actual measures for code generation (280 codes per round; 200 Python, 80 C). C code has a 94% $\rightarrow$ 100% pass rate, which we do not interpret (unvalidated C++ analysis).

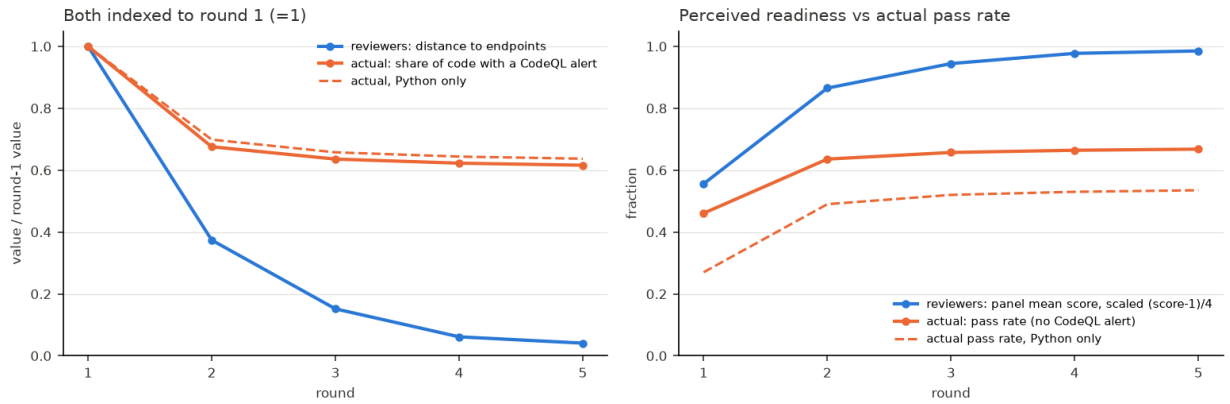


Figure 3: Left: reviewers' distance to the endpoints and the real alert rate, each divided by its round-1 value (same axis, no dual scale). Right: reviewers' panel mean score rescaled to $[0, 1]$ vs. the actual pass rate.

Hiring. Conformity (variance $\approx 2.4\times$ lower) and inertia (the panel mean stops moving after round 2) are clear. Polarization is *not* observed: scores stay near 3.3 and only 1–2% sit at the endpoints. Adversarial influence is visible but small (Fig. 4): ethnicity bias (most minus least privileged group) of the adversary falls from 0.40 to 0.26, while the other three evaluators move from 0.01 to 0.083 by round 3 and *stay at 0.083 after the adversary is removed*; without an adversary the panel bias stays at about 0.02. Gender bias is negligible. These are effects of about 0.08 points on a 5-point scale from a single trial over 320 résumés per condition, so we regard the persistence result as suggestive, not established.

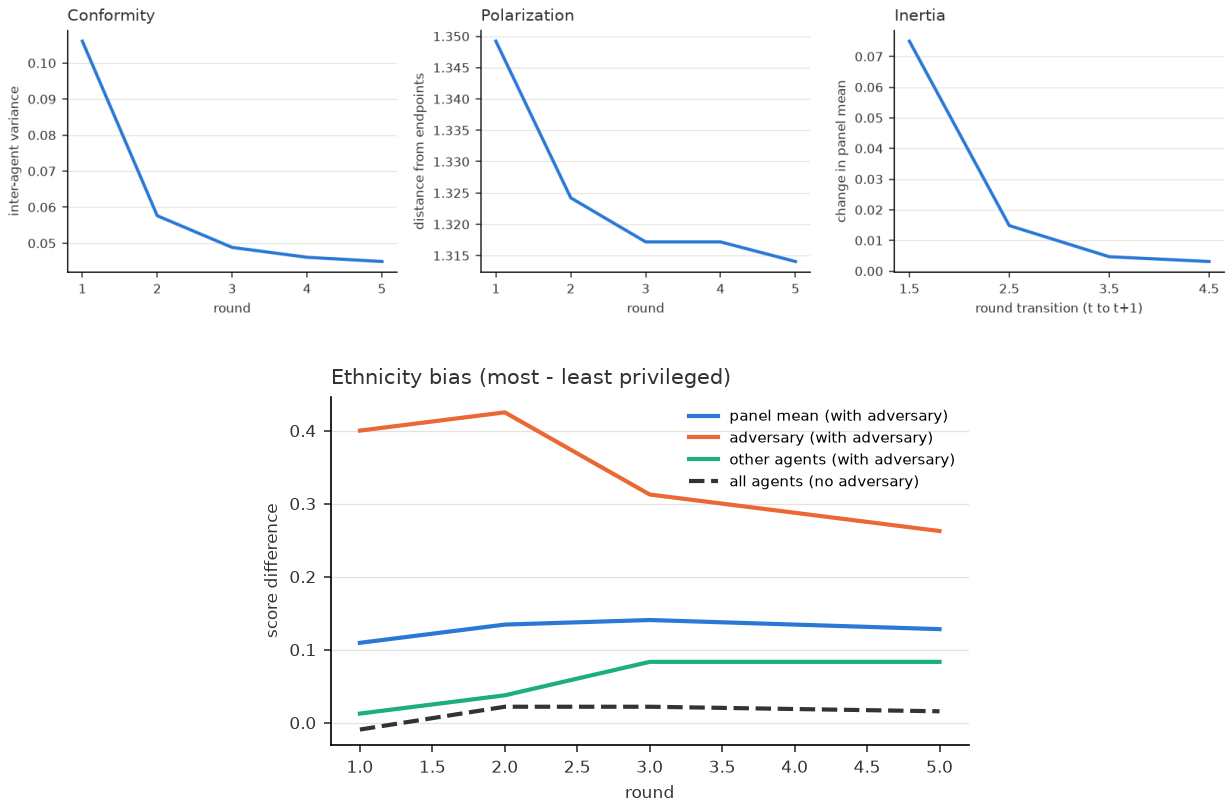


Figure 4: Hiring. Top: homogenization metrics (no adversary; note the zoomed y-axes: the polarization curve spans only 1.31–1.35). Bottom: ethnicity bias with the adversary present in rounds 1–2, and the no-adversary baseline (dashed).

Peer review (partial). Over the three completed rounds the variance across reviewers halves ($0.129 \to 0.063$) and the panel mean stops moving ($0.56 \to 0.14$), while ratings rise for every paper type (Fig. 5; oral $6.1 \to 6.7$, rejected $5.1 \to 5.8$) and ordering by decision is preserved. The paper's *widening disparity across research areas* is not reproduced yet: the spread of area means (14 areas with ≥ 5 papers) goes from 1.14 to 0.87, but with about 20 papers per area and one trial this is noisy; we need all five rounds and more papers.

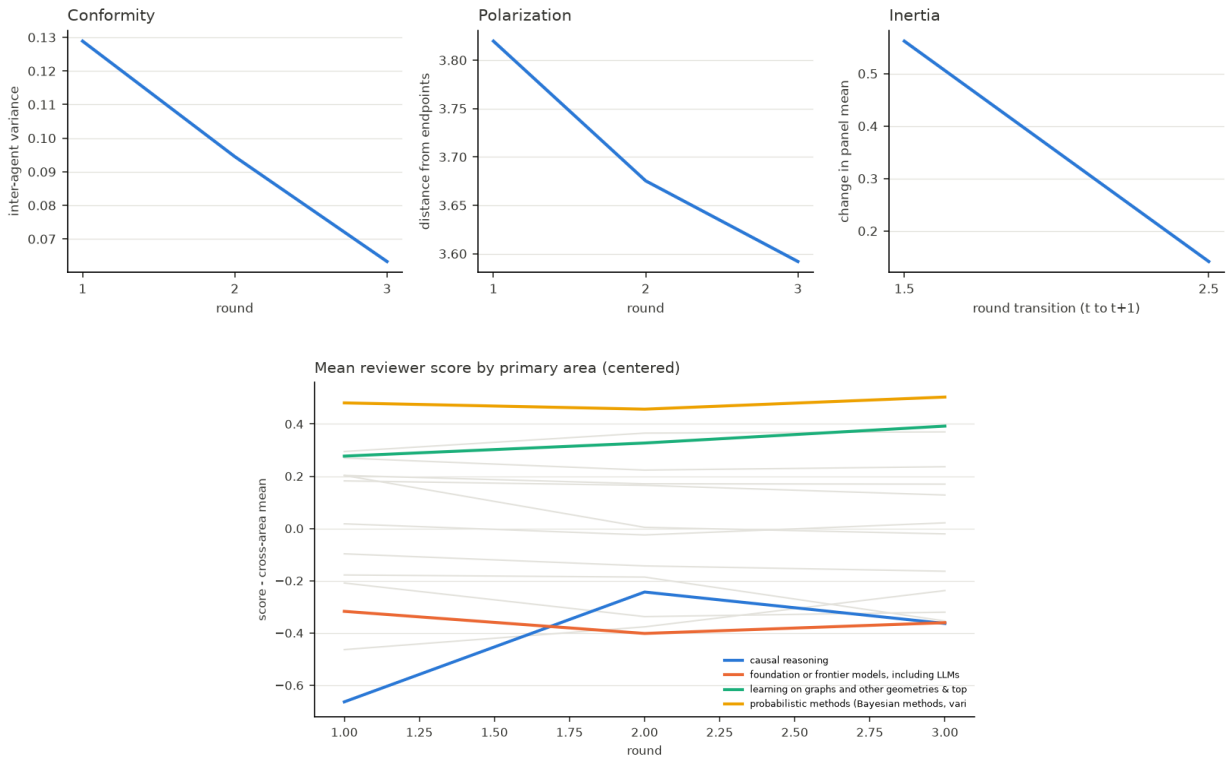


Figure 5: Peer review (3 of 5 rounds, 194 papers). Top: homogenization metrics. Bottom: mean rating by primary area (centered); the four extreme areas are highlighted.

4 Differences from the paper and limitations

- **One model, one trial.** Every agent is Qwen3.8-27B; the paper's 16 mixed-model setups and 10-trial confidence intervals are not reproduced. Cross-trial variance and the code-similarity judge need ≥ 2 trials.
- **Data.** Hiring job descriptions and résumés are generated by Qwen3.8 itself (paper: Gemini) and use one signal variant per group (paper: five); the run uses 40 of 195 job descriptions. Stereotype directions come from BLS CPS Table 11 (ordering by over-representation, which the paper does not specify). For peer review OpenReview is not reachable, so papers come from public mirrors (PDFs text-extracted with PyMuPDF); withdrawn papers have no text available, so "rejected/withdrawn" is filled with rejected only.
- **Ambiguities resolved by us.** Code reviewers see all earlier code and all reviewers' feedback (the paper's wording is ambiguous); the hiring adversary role rotates over the four evaluators by hashing the job description; unparsable replies are re-sampled twice and then dropped.
- **Interpretation.** Score changes in code generation mix real quality improvement with homogenization; the paper's strict-inequality definitions also register ties at the scale ceiling as "no change".
- **Not yet validated:** C/C++ CodeQL analysis.

5 Reproduction

Code and data live in `steering_agents/`. Launch one job per application: `python scripts/launch_mas_pipeline --tier medium`; continue an interrupted run with `--resume <run_dir>`; regenerate figures and tables with `scripts/analyze.py` and `scripts/quality_report.py` (outputs in `<run>/analysis/`). The run described here is `experiments/medium_20261026_234912`; design notes are in `notes/plan.md`.

References

[1] P. Ganesh, K. Wilson, L. Zappella, B.-J. Theobald, N. Apostoloff, L. Monteiro Paes, N. Sivakumar. *Homogenization in Multi-Agent Systems.* arXiv:2610.09824, 2026.